

Tangential Closest Gap Based (TCG) Reactive Obstacle Avoidance Navigation for Cluttered Environments

Muhannad Mujahed, Hussein Jaddu, Dirk Fischer, and Bärbel Mertsching

Abstract—This paper addresses a new obstacle avoidance technique for mobile robots moving in dense and complex environments. Our approach is to integrate two concepts: The Closest Gap (CG) for fetching and analyzing openings surrounding the robot and the Tangential Escape (TE) for reactive obstacle avoidance navigation. This combination results in faster and less oscillatory robot trajectories than the well known approaches designed for complex scenarios, e.g. the Nearness-Diagram Navigation methods. Motion commands are derived with proven stability in the Lyapunov sense for the control system. The power of our method is demonstrated by simulation and experimental results.

I. INTRODUCTION

The increasing need of employing robots in high-risk areas hit by natural disasters has attracted the attention of researchers worldwide to develop fully autonomous mobile robots. The main objective of these robots is to carry out the assigned tasks in places where human presence is dangerous or impossible such as search and rescue, military and exploration.

Usually, a real world disaster environment is unstructured and hazardous. Therefore, traditional motion planning techniques which depend on a predefined map cease to function properly. To overcome this limitation the mobile robot should have the capability to explore the unknown area, automatically generate maps and localize itself within this map. This is achieved by on-board sensing devices detecting instantaneous changes in the environment. In addition, it has to recognize victims and correctly label the map with the victims' positions. In all cases it is necessary to have a reactive obstacle avoidance algorithm to safely reach given goal locations and to deal with dynamic changes such as moving obstacles.

In the literature, there is a rich variety of algorithms that deal with the problem of reactive navigation. In this section, we focus on the most popular techniques showing their advantages and drawbacks.

Early work in this topic includes the Artificial Potential Field APF approaches, initially proposed by Khatib [1]. The main idea stems from the gravitational force field principle. Within this concept, the robot is repelled from obstacles and attracted towards the goal by assuming that opposite forces

are applied on the robot from the goal and surrounding obstacles. The resultant force determines the robots subsequent direction and motion equations. Researchers have come up with a plethora of proposals to enhance this concept (e.g. [2] [3]). While APF methods are computationally efficient, they often fail to find paths between closely spaced obstacles; they also can lead to an oscillatory behavior in narrow corridors.

The approaches defined in [4] [5] use a very simple criteria to reach the goal. The appearance of obstacles in the vicinity of the robot pushes it to escape from the closest one by temporarily switching the desired goal location into a virtual one till passing the risk. The concepts of Artificial Potential Field and Tangential Escape are used for setting the new virtual goal location in [4] and [5], respectively. The latter approach generates smoother robot trajectories than the former one. Although such approaches are very simple to implement and result in faster reactions, they cease to function in slightly complex environments.

Other approaches use an incremental search to plan a path such as: A* and its variants (e.g. [6]), RRT [7] or its recent variant RRT* [8]. These methods are considered computationally expensive and the planned path needs to be recomputed entirely when a dynamic obstacle blocks the path or when the goal location is changed [9]. Several extensions for A* have been developed to speed up the searching process such as: D* Lite [10] which modifies its previous search results locally, and Anytime SIPP [11] which drops the time dimension from the path planning for states after the time horizon. These enhanced versions introduce a faster planner, but are still inefficient for dynamic environments.

We close this section by the obstacle avoidance algorithms designed for cluttered environments. The most popular one is the Nearness-Diagram (ND) [12] which uses a divide and conquer strategy to identify the navigational situation based on the data perceived by sensors, and then generates the corresponding motion law. ND identifies five different navigational laws, whereas the enhanced method named ND+ [13] adds another one. The Smooth Nearness-Diagram (SND) method [14] was then developed which proposes a single motion law to be applicable to all possible configurations of surrounding obstacles. In this regard, the robot trajectory is based on all nearby obstacles, not just the closest two, and this leads to smoother paths. However, a problem of deadlock can appear in the SND in narrow corridors with high threats on one side of the robot and low threats on the other. The Closest Gap (CG) approach proposed in our earlier work [15] overcame this drawback by considering the ratio of threats on the two sides of the robot and providing stricter

This work was supported by the German Academic Exchange Service.

M. Mujahed, D. Fischer and B. Mertsching are with the Dept. of Electrical Engineering and Information Technology, GET-Lab, University of Paderborn, Pohlweg 47-49, D-33098 Paderborn, Germany. {mujahed, fischer, mertsching}@get.uni-paderborn.de

H. Jaddu is with the Faculty of Engineering, Al-Quds University, P.O. Box 20002, Abu Dies, Jerusalem, Palestine. jaddu@eng.alquds.edu

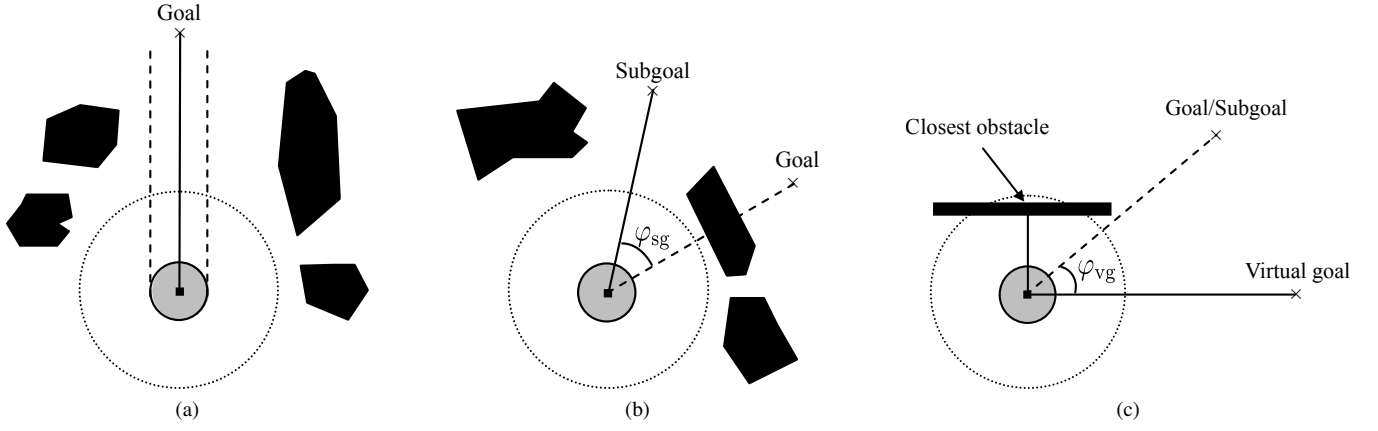


Fig. 1: Situations and their associated actions. (a) Example showing Free-Way and High-Safety situation; no actions is taken. (b) Dangerous-Way and High-Safety situation; the goal location is rotated towards the gap closest to the goal (subgoal). (c) Low-Safety situation, where the goal/subgoal position is switched temporarily to a virtual goal. This forces the robot to navigate parallel to the tangent of the closest obstacle.

deviation against the nearest obstacles. Furthermore, a new algorithm for analyzing gaps was introduced that alleviates oscillations, reduces computational complexity and achieves smoother behavior. Experiments show that these approaches operate well in complex scenarios and avoid local traps. Navigation in such environments with faster speed and less oscillatory behavior is the motivation of our work.

The power of the proposed Tangential Closest Gap (TCG) navigation approach lies in the integration of two concepts: The Closest Gap (CG) [15] for fetching and analyzing openings surrounding the robot and the Tangential Escape (TE) [5] for reactive obstacle avoidance navigation. The TCG method drives the robot towards the goal if the direct path towards it is a collision-free path. If not, the goal is switched to be within the navigable gap closest to it. Whenever the distance to an obstacle gets less than a predefined safe distance during navigation, the goal location is switched again to another point that keeps the robot moving parallel to the tangent of the closest obstacle. The resultant trajectory is faster and less oscillatory when compared to the CG approach. Furthermore, motion commands are derived with proven stability for the control system.

This paper is organized as follows: Section II presents the reactive navigation method design. In sections III and IV, we show the simulation and experimental results. Finally, section V highlights our conclusions.

II. THE REACTIVE NAVIGATION METHOD DESIGN

In this section, the Tangential Closest Gap Navigation (TCG) method for collision avoidance is presented. For simplicity, we assume a circular and holonomic mobile robot. However, an abstraction layer can be introduced to take into account vehicle shape and non-holonomic constraints. For more information about developing such frameworks the reader is directed to [16]. The TCG technique works as follows: First, sensory information is periodically analyzed to determine the distribution of obstacles surrounding the robot. Based on this analysis, the current situation is identified and its associated action is performed as described in subsection

II-A. In subsection II-B, we illustrate how the motion commands are derived from the kinematic model of the mobile robot.

A. Situations and Associated Actions

Navigation in a space occupied by obstacles requires changing the current robot heading to avoid collisions. Therefore, the goal location is rotated to another point temporarily till passing the risk. The rotation angle is calculated based on two criteria:

Criterion 1: Way to goal criterion. We consider two situations: Free-Way and Dangerous-Way. In order to identify the current situation, we construct the configuration space, then we check whether the line segment connecting the robot to the goal intersects any obstacle or not. If so, the situation is a Dangerous-Way. Otherwise, it is a Free-Way. For a Free-Way situation, no change in the goal location occurs (see Fig. 1a). For a Dangerous-Way, the goal position is rotated by an angle φ_{sg} relative to the heading direction of the robot. We refer to the new position as a subgoal, which drives the robot towards the navigable gap closest to the goal (closest gap) as shown in Fig. 1b.

Criterion 2: Safety criterion. The existence or absence of obstacles within a predefined safe distance D_s around the robot (Low-Safety and High-Safety) determines the two situations of this criterion. Only for a Low-Safety situation the goal/subgoal location is rotated by an angle φ_{vg} in order to avoid the risk of collision. Otherwise, no change is performed to the actions associated with criterion 1 situations. We refer to the new position as a virtual-goal (Fig. 1c). Notice that the actions associated with criterion 1 situations have a higher priority than those corresponding to criterion 2.

With each sensor update, the current goal position is denoted by X_{cg} and calculated as:

$$X_{cg} = \begin{bmatrix} \cos(\varphi) & \sin(\varphi) \\ -\sin(\varphi) & \cos(\varphi) \end{bmatrix} X_g \quad (1)$$

where X_g is the goal location and $\varphi = \varphi_{sg} + \varphi_{vg}$ the total rotation angle.

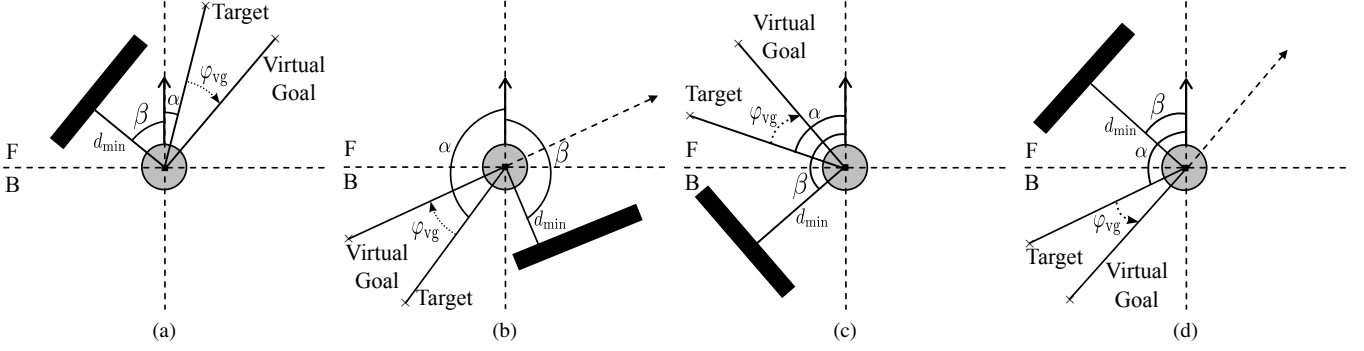


Fig. 2: Different locations for the goal and the closest obstacle relative to the current robot position. (a, b) $\text{sign}(\alpha) \neq \text{sign}(\beta)$ where $|\gamma| < \pi$ for (a) and $|\gamma| \geq \pi$ for (b). (c, d): $\text{sign}(\alpha) = \text{sign}(\beta)$ where $|\beta| \geq |\alpha|$ for (c) and $|\beta| < |\alpha|$ for (d).

Next, we explain how the above mentioned rotation angles are calculated.

Calculating Subgoal Rotation Angle: Setting φ_{sg} depends on the location of the goal and the width of the gap. If the angle towards the goal falls within the closest gap, there is no need to rotate the goal (i.e. $\varphi_{\text{sg}} := 0$). Else, φ_{sg} is set based on the angular width of the closest gap:

$$\varphi_{\text{sg}} = \begin{cases} \theta_{\text{mid}} - \theta_{\text{g}}, & \text{if } \text{dist}(\theta_{\text{cs}}, \theta_{\text{mid}}) < \text{dist}(\theta_{\text{cs}}, \theta_{\text{scs}}) \\ \theta_{\text{scs}} - \theta_{\text{g}}, & \text{otherwise} \end{cases} \quad (2)$$

where θ_{mid} and θ_{scs} are defined as [14]:

$$\theta_{\text{mid}} = \begin{cases} \theta_{\text{cs}} - \frac{(\theta_{\text{cs}} - \theta_{\text{os}})}{2}, & \text{if } \theta_{\text{cs}} \text{ is a left side} \\ \theta_{\text{cs}} + \frac{(\theta_{\text{os}} - \theta_{\text{cs}})}{2}, & \text{otherwise} \end{cases} \quad (3)$$

$$\theta_{\text{scs}} = \begin{cases} \theta_{\text{cs}} - \arcsin\left(\frac{R + D_s}{D_{\text{cs}}}\right), & \text{if } \theta_{\text{cs}} \text{ is a left side} \\ \theta_{\text{cs}} + \arcsin\left(\frac{R + D_s}{D_{\text{cs}}}\right), & \text{otherwise} \end{cases} \quad (4)$$

where D_{cs} and θ_{cs} are the distance and angle towards the obstacle point creating the gap (closest gap) side closer to the goal, respectively, θ_{os} the angle towards the other side of the closest gap, θ_{g} the angle towards the goal and R the robot radius. The dist operator in 2 calculates the minimal angular distance between two angles.

This subgoal rotation angle φ_{sg} will point the robot towards the center of the closest gap if it is narrow. For a wide closest gap, φ_{sg} heads the robot in such a way that D_s is maintained between the gap side closer to the goal and the robot boundary. For determining the closest gap in 2, the reader is directed to [15].

Calculating Virtual Goal Rotation Angle: Setting φ_{vg} is based on the angle towards the obstacle point closest to the robot and the angle towards the goal/subgoal. In the remaining part of this subsection, we call the goal or the subgoal the target based on checking criterion 1 situations. The rotation angle proposed in the TE approach [5] is determined so that the robot takes the direction of the tangent to the obstacle boundary. Always seeking the goal without analyzing free openings (subgoals) makes the TE

method applicable only for simple scenarios. Experiments show that the TE can be trapped in different situations, such as the one explained hereinafter in subsection III, and generates longer trajectories. Moreover, it is not stated in the paper how the leaving condition is set to resume the progress towards the goal after circumnavigating an obstacle boundary or switching to follow the contour of another obstacle when necessary. With the help of generating subgoals in free openings, our approach improves φ_{vg} to get rid of trapping problems. We compute φ_{vg} in such a way that the robot follows a path that is tangent to the obstacle boundary in the direction closer to the target: Let α be the angle towards the target, φ_{vg} is defined as:

$$\varphi_{\text{vg}} = \begin{cases} -\text{sgn}(\beta)\frac{3\pi}{2} - \gamma, & \text{if } |\gamma| \geq \pi \wedge \text{sgn}(\alpha) \neq \text{sgn}(\beta) \\ -\text{sgn}(\beta)\frac{\pi}{2} - \gamma, & \text{if } |\gamma| < \pi \wedge \text{sgn}(\alpha) \neq \text{sgn}(\beta) \\ -\text{sgn}(\beta)\frac{\pi}{2} - \gamma, & \text{if } |\beta| \geq |\alpha| \wedge \text{sgn}(\alpha) = \text{sgn}(\beta) \\ +\text{sgn}(\beta)\frac{\pi}{2} - \gamma, & \text{if } |\beta| < |\alpha| \wedge \text{sgn}(\alpha) = \text{sgn}(\beta) \end{cases} \quad (5)$$

where β is the angle towards the obstacle point closest to the robot and $\gamma = \alpha - \beta$.

Fig. 2 shows different situations and their corresponding virtual rotation angles where the new target location after rotation is labeled Virtual Goal. The rotation angle proposed in [5] drives the robot towards the direction explained by the dashed arrows in Fig. 2c and Fig. 2d. Generating subgoals in the TCG helps in choosing the right escaping (tangent) direction. This is due to the fact that subgoals are always located within a navigable opening in case that no direct free path towards the goal exists.

The leaving condition occurs when the angular distance between α and β exceeds 90° , i.e. we set φ_{vg} calculated above to 0 if $\Psi(\alpha, \beta) > \frac{\pi}{2}$ where $\Psi(\theta_1, \theta_2)$ is defined as follows:

$$\Psi(\theta_1, \theta_2) = \begin{cases} |\theta_1 - \theta_2|, & \text{if } |\theta_1 - \theta_2| < \pi \\ 2\pi - |\theta_1 - \theta_2|, & \text{otherwise} \end{cases} \quad (6)$$

B. Determining Motion Commands

In this subsection, the motion commands (velocities) derived from the kinematic model of the robot are presented. Figure 3 shows a mobile robot moving towards a goal.

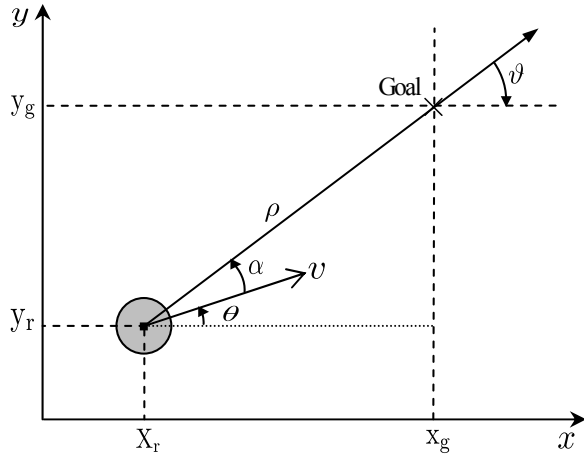


Fig. 3: Position of a mobile robot moving towards a goal.

Its kinematic model in polar coordinates considering two actuator wheels and non-holonomic constraints is [4] [17]:

$$\dot{\rho} = -v \cos(\alpha) \quad (7)$$

$$\dot{\alpha} = -w + v \frac{\sin(\alpha)}{\rho} \quad (8)$$

$$\dot{\vartheta} = -v \frac{\sin(\alpha)}{\rho} \quad (9)$$

where ρ is the distance between the robot and the goal, α the orientation error, and ϑ the angle between the line to the goal and the x axis of the global coordinate system, v the linear velocity and w the angular velocity. The value of ρ in 8 and 9 must not equal zero. Otherwise, the values of $\dot{\alpha}$ and $\dot{\vartheta}$ will be indefinite. Therefore, we assume that the goal is reached when ρ is less than ϵ , where ϵ is any small value.

The objective is to find a suitable state dependence motion law which ensures that the state variables ρ and α go asymptotically to zero. We consider the following Lyapunov function candidate [5]:

$$V = \frac{1}{2}\rho^2 + \frac{1}{2}\alpha^2 \quad (10)$$

The derivative of the Lyapunov function is a negative definite if the motion commands (v, w) are defined as:

$$v = k_b v_{limit} \cos(\alpha) \quad (11)$$

$$w = \text{sat}_{[-w_m, w_m]}(k_m \alpha + \frac{v \sin(\alpha)}{\rho}) \quad (12)$$

These equations are adapted from [5]. The value of k_b is set to $\tanh(\rho)$ if the current situation concerning criterion 1 is Free-Way (refer to subsection II-A). This is to ensure smooth braking when reaching the goal. Otherwise, it is set to 1. In the TE approach, k_b is always set to $\tanh(\rho)$ since it always seeks the real goal (it doesn't generate subgoals as in the TCG). The operator $\text{sat}(x)$ limits x between two boundaries. It is used in 12 to cap the value of w_m between the maximum negative and positive values (w gets greater than w_m when $\rho < 1$). We control the speed by specifying v_{limit} to maintain safety near close obstacles, particularly in

cluttered environments [15]:

$$v_{limit} = (\sqrt{1 - \text{sat}_{[0,1]}(\frac{D_{vs} - d_{min}}{D_{vs}})}) \cdot v_{max} \quad (13)$$

where v_{max} is the maximum linear speed of the robot, D_{vs} a predefined safe distance (called velocity safe distance in [15]), and d_{min} the distance towards the obstacle point closest to the robot boundary. The idea behind using the sat operator in 13 is to cap the velocity at 0 if the robot touches the obstacle and v_{max} if the obstacle is outside D_{vs} . The maximum value of the angular velocity w in 12 corresponds to $\frac{\partial w}{\partial \alpha} = 0$. This lead us to the value of the parameter k_m :

$$k_m = \frac{w_m - \frac{1}{2}k_b v_{limit}}{\frac{\pi}{4}} \quad (14)$$

where w_m is the maximum angular velocity obtained from the data sheet of the robot.

III. SIMULATION RESULTS

This section shows the differences in the execution of the TE, SND, CG and TCG navigation methods. They were implemented using the open-source Player/Stage robot software system. We used a rectangular differential drive robot with a length of 0.53 m and a width of 0.49 m. The maximum translational speed was set to $v_{max} = 0.5 \text{ m/s}$, the maximum rotational one to $w_{max} = 1.0 \text{ rad/s}$ and the safe distance to $D_s = 1 \text{ m}$. The simulated laser rangefinder scans $n = 683$ points over 240° with a maximum range of $d_{max} = 5.6 \text{ m}$. We set the velocity safe distance to $D_{vs} = 1 \text{ m}$ in the CG and TCG methods.

For these simulations, we created a map with narrow corridors where the robot should do sharp turns passing all gaps till reaching the goal. Fig. 4 shows this map where the start and goal locations are also identified.

A. TE Simulation

The trajectory followed using the TE method is shown in Fig. 4a. The robot navigates parallel to the tangent of the closest obstacle until passing the obstacle marked as C. At this moment, the obstacle closest to the robot switches between the right (point D) and left (point F) side of the robot causing sharp trajectory changes. Finally, it comes to a full stop after 75 sec when colliding with obstacles on the side labeled E. This is due to the fact that the TE approach doesn't analyze the laser data to find possible openings. It drives the robot towards the goal, and only if obstacles enter the security zone the robot starts to react against the one closest to it. The closest obstacle can change rapidly between two successive laser scans causing sharp turns. In addition, the robot can face a U-shaped obstacle resulting in such a dead end situation. If a subgoal is generated inside the gap labeled Gap on the figure (as in the TCG method), the robot will stop following the contour of obstacles and directly move towards it whenever the leaving condition is satisfied. The recorded motion commands versus time are shown in Fig. 4e.

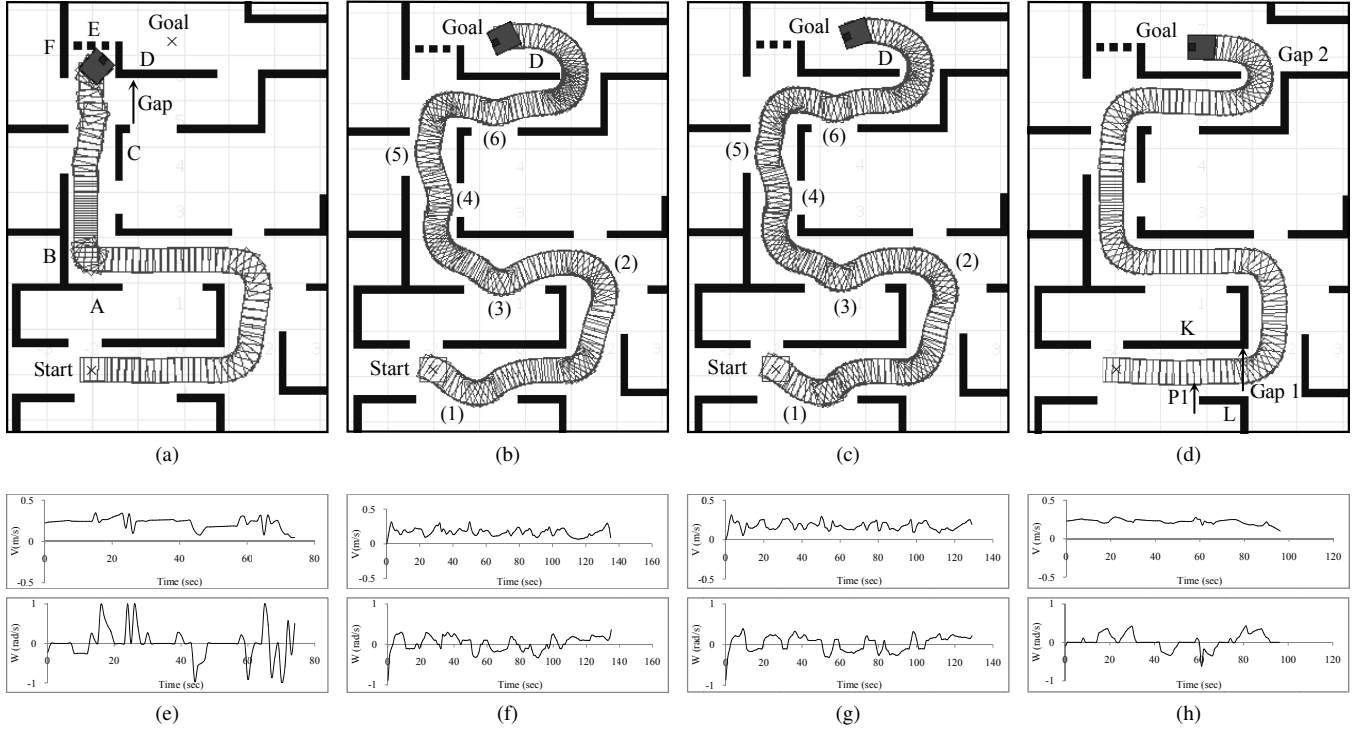


Fig. 4: Simulations. (a - d) Trajectory followed by (a) TE, (b) SND, (c) CG and (d) TCG methods. (e - h) Translational and rotational velocities versus time for (e) TE, (f) SND, (g) CG and (h) TCG methods.

B. SND and CG Simulations

The path followed using SND and CG methods is shown in Fig. 4b and Fig. 4c, respectively. No actual differences are observed in the trajectories of the two approaches except at one point where the robot gets very close to obstacles on the side labeled D running the SND algorithm. Moreover, the goal was reached in 135 sec using the SND method and 129 sec using the CG approach. The robot is deflected meaninglessly towards free areas while progressing towards the goal. This appears clearly at the points labeled 1 - 6. This is due to the fact that avoiding nearby obstacles in the ND, SND and CG methods is based on the artificial potential fields, i.e. each obstacle deflects the robot heading direction by an angle ranging from 0 to 180 degrees. The value of this angle depends on the proximity of the obstacle to the robot. Fig. 4f and Fig. 4g show the recorded motion commands (v and w) versus time for the SND and CG methods.

C. TCG Simulation

Fig. 4d shows the trajectory followed by the TCG method. At the starting point the closest obstacle is on the side labeled K and the subgoal is towards the center of Gap 1. The angular distance between them ($\Psi(\alpha, \beta)$ in 6) is approximately $\frac{\pi}{2}$. Therefore, the leaving condition is satisfied and the progress will be directly towards the subgoal. When the robot reaches point P1, the closest obstacle is on the side labeled L. The angular distance between this obstacle and the subgoal is less than $\frac{\pi}{2}$. Therefore, the robot navigates parallel to the tangent of obstacles on side L until the leaving condition is satisfied

again. This process is repeated until having passed all gaps and finally having reached the goal. One can notice from the trajectory near Gap 2 that the problem of getting close to the side containing fewer threats (obstacles) as in the SND method does not appear here. The goal is reached in 95 sec only. The TCG method drives the robot much faster with a better trajectory than the previously mentioned methods. This can be noticed from the density of bounding boxes on the map and from plotting the recorded motion commands versus time as shown in Fig. 4h.

IV. EXPERIMENTAL RESULTS

We tested the TCG method on our rescue robot (GETbot); a skid-steering Pioneer 3-AT mobile robot. The specifications of the laser scanner are the same as the simulated one introduced in section III. The mobile robot has a rectangular shape (0.53×0.49 meters) and non-holonomic constraints. The maximum travel speed is $0.7 m/s$ and the maximum rotational one is $140^\circ/s$. These velocities were limited to ($0.3 m/s, 35^\circ/s$) while carrying out the experiments. The safe distance and the velocity safe distance were set to $1 m$. We followed the solution proposed in [16] to adapt our algorithm to accommodate the shape constraint.

We ran the TCG-controlled Pioneer 3-AT through difficult courses. Fig. 5 shows one of our experiments where none of the obstacle locations was known to the robot in advance. In this scenario, the obstacles are boxes randomly distributed along a corridor as shown in the figure. We recorded the linear and angular velocities (v and w) over the course of this experiment. Fig. 5b shows the velocities for the TCG method;

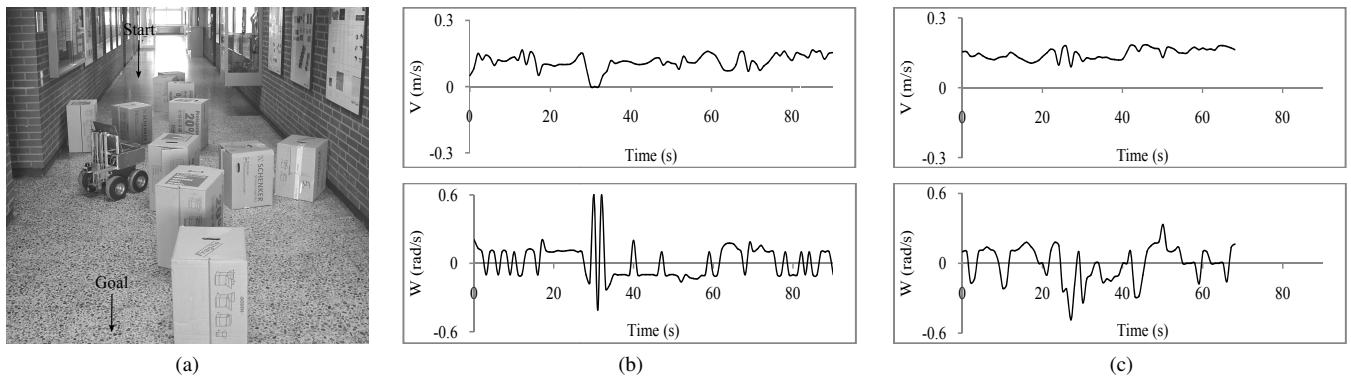


Fig. 5: Experimental results. (a) Experimental setup. (b) Velocity profile for the CG approach. (c) velocity profile for the TCG approach.

they are plotted against the time elapsed by the robot. The experiment was carried out again using the CG method. The recorded velocities versus time for the CG method are shown in Fig. 5c as well. One can notice that the TCG generated less oscillatory trajectory than the CG method. Furthermore, the goal was reached in 68 sec using the TCG method while it took approximately 90 sec for the CG.

V. CONCLUSIONS

We have presented the Tangential Closest Gap (TCG) Navigation method for complex environments. This approach drives the robot faster with less oscillatory motion than the well known approaches designed to operate in such environments. This is achieved by combining two methods: the Closest Gap (CG) and the Tangential Escape (TE) navigation methods. The system is controlled by choosing motion commands that are stable in the Lyapunov sense. Comparisons between previous works and the TCG in simulations and experiments demonstrated the advantages of this combination.

The TCG solves the motion problem using only the current sensor data as explained in section II. Therefore, trap situations or cyclic motions may arise. To solve such a problem, the TCG should be used in conjunction with a planner (a hybrid system) such as those proposed in [18] and [19]. In this regard, these undesirable situations can be avoided while still using a high-frequency navigation module which is necessary in unstructured and dynamic environments.

Future work on the TCG includes adapting our controller to take into account navigation over uneven terrains and dealing with sliding problems.

REFERENCES

- [1] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *Int. J. Rob. Res.*, vol. 5, pp. 90–98, Spring 1986.
- [2] L. Montano and J. R. Asensio, "Real-time robot navigation in unstructured environments using a 3d laser rangefinder," in *IEEE/RSJ Int. Conf. on Intelligent Robots and Systems*, pp. 526 – 532, 1997.
- [3] J. Borenstein, Y. Koren, and S. Member, "The vector field histogram - fast obstacle avoidance for mobile robots," *IEEE Transactions on Robotics and Automation*, vol. 7, pp. 278–288, 1991.
- [4] C. R. Secchi, H. and M. V., "Discrete stable control of mobile robots with obstacles avoidance," in *Proc. of the 11th International Conference on Advanced Robotics (ICAR)*, pp. 405 – 411, 2001.
- [5] A. Ferreira, F. G. Pereira, R. F. Vassallo, M. Sarcinelli-Filho, and T. F. Bastos-Filho, "An approach to avoid obstacles in mobile robot navigation: The tangential escape," in *SBA. Sociedade Brasileira de Automatica*, vol. 19, pp. 395–405, 2008.
- [6] D. Dolgov, S. Thrun, M. Montemerlo, and J. Diebel, "Path planning for autonomous vehicles in unknown semi-structured environments," *Int. J. Rob. Res.*, vol. 29, no. 5, pp. 485–501, 2010.
- [7] Y. Kuwata, G. A. Fiore, J. Teo, E. Frazzoli, and J. P. How, "Motion planning for urban driving using RRT," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, (Nice, France), pp. 1681–1686, September 2008.
- [8] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *International Journal of Robotics Research*, vol. 30, pp. 846–894, June 2011.
- [9] J. Park, C. Johnson, and B. Kuipers, "Robot navigation with model predictive equilibrium point control," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 4945–4952, 2012.
- [10] S. Koenig and M. Likhachev, "Fast replanning for navigation in unknown terrain," *IEEE Trans. on Robo.*, vol. 21, pp. 354–363, 2005.
- [11] V. Narayanan, M. Phillips, and M. Likhachev, "Anytime safe interval path planning for dynamic environments," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, October 2012.
- [12] J. Minguez and L. Montano, "Nearness diagram (ND) navigation: collision avoidance in troublesome scenarios," *IEEE Transactions on Robotics and Automation*, vol. 20, no. 1, pp. 45–59, 2004.
- [13] J. Minguez, J. Osuna, and L. Montano, "A "divide and conquer" strategy based on situations to achieve reactive collision avoidance in troublesome scenarios," in *IEEE International Conference on Robotics and Automation*, 2004.
- [14] J. Durham and F. Bullo, "Smooth nearness-diagram navigation," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 690–695, September 2008.
- [15] M. Mujahed, D. Fischer, B. Mertsching, and H. Jaddu, "Closest Gap Based (CG) Reactive Obstacle Avoidance Navigation for Highly Cluttered Environments," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 1805 – 1812, October 2010.
- [16] J. Minguez and L. Montano, "Extending collision avoidance methods to consider the vehicle shape, kinematics, and dynamics of a mobile robot," *IEEE Trans. on Rob.*, vol. 25, no. 2, pp. 367–381, 2009.
- [17] M. Aicardi, G. Casalino, A. Bicchi, and A. Balestrino, "Closed loop steering of unicycle-like vehicles via lyapunov techniques," *IEEE Robotics and Automation Magazine*, vol. 2, pp. 27–35, 1995.
- [18] N. S. J. Minguez, L. Montano and R. Alami, "global nearness diagram navigation (gnd)," in *IEEE Int. Conf. on Robotics and Automation*, (Seoul, Korea), May 2001.
- [19] C. Stachniss and W. Burgard, "An integrated approach to goal-directed obstacle avoidance under dynamic constraints for dynamic environments," in *IEEE/RSJ Int. Conf. on Intelligent Robots and Systems*, (Switzerland), pp. 508 – 513, 2002.